CLOUD-04: AWS Lambda & Serverless Monitoring

> **Series:** CLOUD | **Notebook:** 4 of 8 | **Created:** March 2026 | **Last Updated:** 04/04/2026

Overview

This notebook covers serverless monitoring with Dynatrace, focusing on AWS Lambda. You will learn how to monitor Lambda function performance (cold starts, duration, errors, throttles), integrate API Gateway tracing, analyze Step Functions workflows, assess DynamoDB performance, and build end-to-end serverless application tracing.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`metrics.read`, `entities.read`, `spans.read`
AWS Integration	AWS cloud integration configured (CLOUD-02)
Lambda Functions	At least one Lambda function with Dynatrace layer or OneAgent extension
Prior Knowledge	CLOUD-01 fundamentals, CLOUD-02 AWS integration

1. Lambda Monitoring Fundamentals

Dynatrace monitors Lambda functions through two complementary approaches:

Approach	Data Source	What It Provides
Cloud Integration	CloudWatch metrics via ActiveGate	Invocations, duration, errors, throttles, concurrent executions
Dynatrace Lambda Layer	OneAgent in Lambda runtime	Distributed traces, code-level visibility, custom metrics

Key Lambda Metrics

Metric	Description	Healthy Range
`cloud.aws.lambda.invocations`	Total invocations	Application-dependent
`cloud.aws.lambda.duration`	Execution time (ms)	< function timeout
`cloud.aws.lambda.errors`	Execution errors	0 (ideally)

Metric	Description	Healthy Range
`cloud.aws.lambda.throttles`	Throttled invocations	0
`cloud.aws.lambda.concurrentExecutions`	Concurrent runs	< reserved concurrency
`cloud.aws.lambda.unreservedConcurrentExecutions`	Unreserved concurrency	< account limit

List Monitored Lambda Functions

```
// List all monitored Lambda functions with runtime and code size
fetch dt.entity.aws_lambda_function
| fieldsKeep id, entity.name, awsLambdaFunctionRuntime, awsCodeSize, tags
| sort entity.name asc
| limit 25

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes AWS_LAMBDA_FUNCTION
// | fieldsKeep id, name, awsLambdaFunctionRuntime, awsCodeSize, tags
// | sort name asc
// | limit 25
```

Lambda Execution Time Over Time

```
// Lambda average execution duration over the last 6 hours
timeseries avgDuration = avg(cloud.aws.lambda.duration), from:-6h, by:
{dt.entity.aws_lambda_function}
| fieldsAdd avgDurationValue = arrayAvg(avgDuration)
| sort avgDurationValue desc
| limit 10
```

2. Cold Start Detection

Cold starts occur when Lambda creates a new execution environment. They add latency (100ms to several seconds depending on runtime and package size).

Understanding Cold Starts

Factor	Impact on Cold Start
Runtime	Java/C# > Python/Node.js
Package size	Larger = slower initialization
VPC attachment	Adds ENI creation time (~1-2s)
Provisioned concurrency	Eliminates cold starts (at cost)
SnapStart	Reduces Java cold starts to ~200ms

Detecting Cold Starts via Init Duration

CloudWatch provides an `InitDuration` metric for cold starts. With Dynatrace Lambda Layer, you can detect cold starts from spans.

```
// Detect Lambda cold starts from spans (init phase)
fetch spans, from:-6h
| filter contains(span.name, "lambda") and isNotNull(faas.coldstart)
| fieldsKeep timestamp, span.name, duration, faas.coldstart,
faas.invocation_id
| filter faas.coldstart == true
| summarize cold_start_count = count(), avg_duration_ms = avg(duration) /
1000000.0
```

Lambda Invocations with Duration Percentiles

```
// Lambda duration percentiles over the last 6 hours
timeseries p50 = percentile(cloud.aws.lambda.duration, 50), from:-6h, by:
{dt.entity.aws_lambda_function}
| fieldsAdd p50Value = arrayAvg(p50)
| sort p50Value desc
| limit 10
```

3. Lambda Error Analysis

Lambda errors fall into two categories:

Error Type	Description	Metric
Function errors	Unhandled exceptions, timeout, OOM	`cloud.aws.lambda.errors`
Invocation errors	Permission issues, throttling, invalid payload	`cloud.aws.lambda.throttles`

Error Rate by Function

```
// Lambda error count by function over the last 24 hours
timeseries errors = sum(cloud.aws.lambda.errors), from:-24h, by:
{dt.entity.aws_lambda_function}
| fieldsAdd totalErrors = arraySum(errors)
| filter totalErrors &gt; 0
| sort totalErrors desc
| limit 10
```

Error-to-Invocation Ratio

```
// Error rate percentage by function over the last 24 hours
timeseries errors = sum(cloud.aws.lambda.errors, default:0), from:-24h, by:
{dt.entity.aws_lambda_function}
| append [
    timeseries invocations = sum(cloud.aws.lambda.invocations, default:1),
from:-24h, by:{dt.entity.aws_lambda_function}
  ]
| fieldsAdd totalErrors = arraySum(errors), totalInvocations =
arraySum(invocations)
| fieldsAdd error_pct = totalErrors / totalInvocations * 100.0
| filter totalErrors &gt; 0
| sort error_pct desc
| limit 10
```

4. Throttle Analysis

Throttling occurs when Lambda cannot allocate an execution environment, typically due to:
- **Account-level concurrency limit** (default 1,000 per region)

- **Reserved concurrency** on the function being exhausted
- **Burst limit** exceeded (3,000 in US regions, varies by region)

Throttled Invocations Over Time

```
// Lambda throttles over the last 24 hours by function
timeseries throttles = sum(cloud.aws.lambda.throttles), from:-24h, by:
{dt.entity.aws_lambda_function}
| fieldsAdd totalThrottles = arraySum(throttles)
| filter totalThrottles &gt; 0
| sort totalThrottles desc
```

Concurrent Executions Trend

```
// Concurrent Lambda executions over the last 6 hours
timeseries concurrency = max(cloud.aws.lambda.concurrentExecutions),
from:-6h, by:{dt.entity.aws_lambda_function}
| fieldsAdd peakConcurrency = arrayMax(concurrency)
| sort peakConcurrency desc
| limit 10
```

5. API Gateway Integration

AWS API Gateway is a common front-end for Lambda functions. Dynatrace monitors the API Gateway-to-Lambda path for end-to-end latency.

API Gateway Monitoring Points

Metric	What It Measures
Count	Total API requests
Latency	End-to-end latency (API GW → Lambda → response)
IntegrationLatency	Lambda execution time only
4XXError	Client-side errors
5XXError	Server-side errors (Lambda failures)

Tracing API Gateway Requests

With the Dynatrace Lambda Layer, traces propagate through API Gateway into Lambda:

```
Client → API Gateway → Lambda Function → DynamoDB / SQS / etc.
   |         |              |                |
   span      span           span             span
   \_________\______________\________________/
              Complete distributed trace
```

```
// Trace spans from API Gateway and Lambda in the last hour
fetch spans, from:-1h
| filter span.kind == "server"
| filter contains(span.name, "lambda") or contains(span.name, "api-gateway")
| fieldsKeep timestamp, trace.id, span.name, duration, span.status_code
| sort timestamp desc
| limit 20
```

6. Step Functions Tracing

AWS Step Functions orchestrate Lambda functions into workflows. Dynatrace can trace the entire state machine execution.

Step Functions Monitoring

What to Monitor	Why
Execution duration	Detect slow workflows
Failed executions	Identify error patterns
State transitions	Find bottleneck states
Lambda cold starts within workflows	Impact on workflow latency

Step Function Execution Tracking via Logs

```
// Step Function execution logs (if forwarded to Dynatrace)
fetch logs, from:-24h
| filter contains(content, "StepFunction") or contains(content,
"StateMachine")
| fieldsKeep timestamp, content, log.source
| sort timestamp desc
| limit 20
```

7. DynamoDB Performance

DynamoDB is frequently used with Lambda for serverless data storage. Key monitoring areas:

Metric	Concern
Read/Write capacity	Throttling if exceeded
Latency	Slow queries impacting Lambda duration
Consumed vs provisioned	Over/under-provisioning
Error count	Conditional check failures, throttles

DynamoDB Span Analysis

```
// DynamoDB call duration from Lambda spans in the last 6 hours
fetch spans, from:-6h
| filter span.kind == "client" and isNotNull(db.system)
| filter db.system == "dynamodb"
| summarize avg_duration_ms = avg(duration) / 1000000.0, call_count =
count(), by:{db.namespace, db.operation}
| sort avg_duration_ms desc
| limit 15
```

8. End-to-End Serverless Tracing

A complete serverless application might follow this pattern:

```
API Gateway → Lambda (auth) → Lambda (business logic) → DynamoDB
                                                      → SQS → Lambda (async processor)
                                                      → SNS → Email notification
```

Dynatrace traces the entire flow as a single distributed trace when the Lambda Layer propagates context headers.

Trace Analysis Across Services

```
// Slowest traces involving Lambda functions in the last hour
fetch spans, from:-1h
| filter span.kind == "server"
| summarize trace_duration = max(duration), span_count = count(), by:
{trace.id}
| sort trace_duration desc
| limit 10
```

Serverless Monitoring Best Practices

Practice	Description
Always install Dynatrace Lambda Layer	Enables distributed tracing and code-level visibility
Set meaningful function names	Avoid auto-generated names for better identification
Monitor concurrency headroom	Alert before hitting account limits
Track cold start ratio	High ratio indicates scaling or provisioned concurrency needs
Correlate API Gateway with Lambda	End-to-end latency matters more than Lambda duration alone

9. Summary and Next Steps

Key Takeaways

- Lambda monitoring combines **CloudWatch metrics** (invocations, errors, throttles) with **Dynatrace Lambda Layer** (traces, code-level visibility)
- **Cold starts** can be detected via spans and mitigated with provisioned concurrency or SnapStart
- **Error-to-invocation ratio** is a more meaningful health metric than raw error count
- **End-to-end tracing** through API Gateway, Lambda, and downstream services requires context propagation

Next Steps

- **CLOUD-05: Azure Integration** — Azure-specific monitoring patterns
- **CLOUD-07: CloudWatch Log Ingestion** — Forward Lambda logs to Dynatrace
- **CLOUD-08: Multi-Cloud Patterns** — Unified serverless monitoring across providers

---